# AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

## Chapter 3: Building AI Workflows with MCP

### Rebuilding the Workflow with Live Data

*Run the launch review against Orbit Labs' systems through MCP, then watch the answer change when the data does*

In Chapter 2, every launch review depended on documents someone had written and pasted in. In this video you run the same review with no documents at all. The agent pulls milestones, tasks, and support tickets straight from Orbit Labs' systems through the MCP server you connected in the previous video.

#### Before you start

You need the `orbit` MCP server connected and the Skill parked, as at the end of video 03_01. If you're not sure, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 03_01
```

Then reset the company data, from a terminal in `orbit-backend`:

```
npm run reset
```

Start a new Claude Code session in `orbit-launch` and confirm with `/mcp` that `orbit` is connected.

#### What's different this time

- **No pasted documents.** The request doesn't reference anything in `inputs/`.
- **No Skill.** The agent has access to the data but none of the team's rules. That's deliberate, and you'll see why in video 03_04.
- **A short request.** It names the task and the source, and nothing else.

#### Step 1: Run the review on live data

Send:

```
Using only the orbit server, review the Orbit 2.0 launch and tell me the
biggest risks.
```

The agent will ask permission for each tool call. All of them read data, so approve them. To save time, you can choose the option that stops Claude Code asking again for that tool. Don't do this for `create_risk_item`, the one tool that writes.

### Step 2: Watch how the agent gathers data

Before reading the answer, look at the sequence of tool calls. A typical run looks something like this:

1. `get_project` to find the launch date and review date.
2. `list_milestones` to see which milestones are at risk.
3. `list_tasks`, often filtered to blocked tasks or tasks with no owner.
4. `search_tickets`, often filtered to open tickets, high priorities, or tickets open for a while.
5. Sometimes `get_ticket` for details on a specific ticket.

Nobody told the agent this plan. It worked it out from the request and from each tool's name, description, and inputs, then chose its own filters. The exact order and filters can differ in your run.

### Step 3: Read what live data reveals

Look for these findings:

- **Duplicate charges:** four open **P1** tickets (TCK-1987, TCK-1992, TCK-2003, TCK-2010), open between 8 and 13 days. The week 40 status update called this "a couple of payment reports."
- **PayGate:** task T-104 is blocked. Its notes say certification moved to November 2 and the team needs about three weeks after that.
- **Pen test:** task T-117 still has no owner and is due October 9.
- **Knock-on effects:** the load test (T-125) can't start until the checkout integration is done.

Every finding points to a ticket or task ID. You can trace each one back to the system it came from, which a pasted summary can't offer.

### Step 4: Change the world

Now simulate something happening after the review. Open a **second terminal** in `orbit-backend` and run:

```
npm run demo:new-ticket
```

This adds a new P1 ticket: six beta customers can't sign in through SSO.

Back in Claude Code, without clearing the conversation, send:

```
Check the support desk again. Anything new that changes the picture?
```

The agent searches the tickets again and finds **TCK-2031**. Nobody wrote a status update or pasted anything in. The answer changed because the system changed.

### Step 5: Save the report

Send:

```
Update your review with anything new and save it to reports/mcp-only.md
```

Approve the file write and check that `reports/mcp-only.md` exists. You'll compare it with `reports/skill-only.md` in the next video. Leave the new ticket in place for now.

### Look at the format, too

Before moving on, look at how the report is structured: its headings, the words used for severity, whether there's a table, what each row includes. All of that was decided by the agent in this run, because nothing told it otherwise. Keep that in mind for video 03_04.

### What changed compared with Chapter 2

- **Freshness.** The answer reflects the systems as they are now, not as they were when someone last wrote an update.
- **No copy and paste.** Removing the manual step also removed the place where four P1 tickets became "a couple of reports."
- **Traceable evidence.** Ticket and task IDs point back to the source of truth.
- **A changing world.** When the data changed, a follow-up question was enough to see it.

The agent completed the workflow, from gathering data to a saved report, using external systems instead of documents.

### If you get stuck

- If the agent can't find the `orbit` tools, run `/mcp`. If the server isn't connected, start a new session from the `orbit-launch` folder.
- If the agent reads files in `inputs/` instead of calling tools, clear the conversation and repeat the request with "using only the orbit server."
- If `npm run demo:new-ticket` says the ticket already exists, run `npm run reset`, then run the demo again.